20. SSH Basics: Keys, Config, Jump Hosts

20. SSH Basics: Keys, Config, Jump Hosts

Phase: Phase 9 — Security and SSH
Estimated time: 1–2 days
Slug: `20-ssh-basics`

Learning objectives

- Generate and use SSH keys
- Write ~/.ssh/config Host entries
- Disable password auth conceptually
- Use jump hosts / ProxyJump

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

SSH is how you reach nearly every Linux server. Passwords work, but keys are stronger, scriptable, and can be revoked per device. A key pair also makes Git hosting, deployment tools, and automation possible without typing secrets. The catch is that OpenSSH is strict about file permissions: if your private key is readable by others it refuses to use it, with a loud warning. That is why `ssh_key_perms` returns 600.

This lesson covers generating keys, installing the public key, the client config file for short host aliases, and the permission rules that cause most “why does SSH ignore my key” tickets.

Core concepts

File	Purpose	Required mode
`~/.ssh/`	Directory for SSH data	`700`
`~/.ssh/id_ed25519`	Private key (keep secret)	`600`
`~/.ssh/id_ed25519.pub`	Public key (shareable)	`644`
`~/.ssh/authorized_keys`	Public keys allowed to log in	`600` or `644`
`~/.ssh/config`	Client aliases and options	`600`
`~/.ssh/known_hosts`	Fingerprints of servers you trust	`644`

Octal modes: 600 means owner read and write, nobody else anything.

Key based login:

```
client                                  server
------                                  ------
id_ed25519      (private, 600)          ~/.ssh/authorized_keys
id_ed25519.pub  (public)  --ssh-copy-id--> contains the public key
      |                                         |
      |  1. connect, offer public key           |
      |---------------------------------------->|
      |  2. server sends challenge              |
      |<----------------------------------------|
      |  3. client signs with private key       |
      |---------------------------------------->|
      |  4. server verifies with public key -> login
```

Worked examples

Generate a modern key pair:

```
ssh-keygen -t ed25519 -C "ada@laptop" -f ~/.ssh/id_ed25519
```

Sample output:

```
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Your identification has been saved in /home/ada/.ssh/id_ed25519
Your public key has been saved in /home/ada/.ssh/id_ed25519.pub
SHA256:Hq3...x0 ada@laptop
```

Check and fix permissions:

```
ls -l ~/.ssh
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519 ~/.ssh/config
stat -c '%a %n' ~/.ssh/id_ed25519
```

Sample output:

```
-rw------- 1 ada ada 411 Oct 21 10:00 id_ed25519
-rw-r--r-- 1 ada ada  96 Oct 21 10:00 id_ed25519.pub
600 /home/ada/.ssh/id_ed25519
```

Install the public key on a server and log in:

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub ada@203.0.113.10
ssh ada@203.0.113.10 'hostname; whoami'
```

Sample output:

```
web01
ada
```

Client config with an alias:

```
cat >> ~/.ssh/config <<'EOF'
Host web
  HostName 203.0.113.10
  User ada
  IdentityFile ~/.ssh/id_ed25519
  Port 22
EOF
ssh web hostname
```

Sample output:

```
web01
```

What happens with loose permissions:

```
chmod 644 ~/.ssh/id_ed25519
ssh web
```

Sample output:

```
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
Permissions 0644 for '/home/ada/.ssh/id_ed25519' are too open.
This private key will be ignored.
```

Helper from the exercise: `ssh_key_perms() { echo 600; }`

Mini-lab

1. Create a lab key: `ssh-keygen -t ed25519 -f ~/.ssh/lab_key -N ""`.
2. Run `stat -c '%a %n' ~/.ssh/lab_key*` and confirm 600 and 644.

3. Break it with `chmod 644 ~/.ssh/lab_key`, then try `ssh -i ~/.ssh/lab_key localhost` and read the warning. Fix it.
4. Add a `Host lab` block to `~/.ssh/config` for localhost or a VM.
5. Run `ssh -v lab` and find the line where your key is offered.
6. Use `ssh-add -l` to see if an agent holds keys; add one with `ssh-add`.
7. View a server fingerprint with `ssh-keyscan localhost` and compare to `~/.ssh/known_hosts` entries.
8. Add a passphrase to a key with `ssh-keygen -p -f ~/.ssh/lab_key`.

Decision guide / failure modes

Symptom	Likely cause	Fix
`Permission denied (publickey)`	Key not in `authorized_keys`	`ssh-copy-id`, check `-v` output
`UNPROTECTED PRIVATE KEY FILE`	Mode wider than 600	`chmod 600 key`
Server still asks password	Home or `.ssh` mode too open on server	`chmod 700 ~/.ssh`, home not group-writable
`REMOTE HOST IDENTIFICATION HAS CHANGED`	Rebuilt server or MITM	Verify, then `ssh-keygen -R host`
Wrong key used	Many keys offered	`IdentitiesOnly yes` in config
Connection timeout	Firewall or wrong port	`nc -vz host 22`

Use ed25519 unless a legacy system needs RSA 4096. Protect private keys with a passphrase and an agent, never copy them between machines casually, and never share anything except the `.pub` file.

Common mistakes

- Sharing private keys
- chmod 777 on .ssh
- Password auth on internet-facing SSH

Exercises

Work in `exercises/20-ssh-basics/`. Read that folder's `README.md` fully before coding.

```
cd exercises/20-ssh-basics
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement ssh_key_perms in exercise.sh. It takes no arguments and must print the octal permission mode that OpenSSH expects for a private key file, with nothing else on the line. The test checks the exact string. Afterwards, create a throwaway key with ssh-keygen, confirm its mode with stat -c '%a %n', deliberately loosen it with chmod 644, and read the warning ssh prints. Restore the correct mode and note why a group-readable private key is a security problem.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/20-ssh-basics/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. ed25519 purpose?
2. What is ProxyJump?
3. Recommended .ssh private key perms?

Further reading

- https://www.openssh.com/manual.html